

Analisis Kerentanan Kriptografis GoBalance

Pemulihan kunci identitas master dari descriptor publik pada jalur kunci berformat Tor, dan kaitannya dengan pengambilalihan domain onion dua forum besar.

Laporan Riset Keamanan Defensif

Repositori objek analisis: gitlab.com/n0tr1v/gobalance (master & patch1)

8 Oktober 2026

Daftar Isi

1. Ringkasan Eksekutif	1
2. Latar Belakang: OnionBalance dan GoBalance	2
3. Pemetaan Jalur Kunci Kriptografis	3
4. Kerentanan Kritis: Nonce Deterministik Konstan	4
4.1 Akar masalah pada kode	4
4.2 Aljabar pemulihan kunci	5
4.3 Karakteristik serangan	5
5. Validasi Proof-of-Concept	6
6. Temuan Sekunder	6
7. Rekonstruksi Skenario Pengambilalihan Domain	7
8. Rekomendasi Remediasi	8
9. Lingkup, Etika, dan Batasan	9

1. Ringkasan Eksekutif

Laporan ini menyajikan hasil analisis keamanan terhadap repositori publik **GoBalance** (gitlab.com/n0tr1v/gobalance), sebuah implementasi ulang dalam bahasa Go dari OnionBalance, yaitu alat penyeimbang beban (load balancer) untuk Tor hidden service v3. Analisis dilakukan untuk menindaklanjuti dugaan keterkaitan antara penggunaan alat ini dengan insiden pengambilalihan (takeover) domain onion yang menimpa dua forum besar beberapa hari terakhir, di mana kedua alamat forum tersebut dialihkan ke domain onion lain yang bukan milik operator aslinya. Fokus analisis adalah mencari kelemahan pada kode yang secara teknis memungkinkan pihak luar menguasai alamat onion tanpa perlu menyentuh server backend maupun infrastruktur operator.

Hasil utama analisis menemukan satu kerentanan kriptografis ber tingkat keparahan **kritis** pada jalur penandatanganan buta (blinded signing) ketika kunci master layanan dimuat dalam format kunci Tor (*hs_ed25519_secret_key*). Pada jalur tersebut, nilai nonce Ed25519 yang dipakai untuk menandatangani sertifikat kunci penandatanganan descriptor menjadi **deterministik dan dapat dihitung oleh siapa pun**, karena komponen rahasia nonce (PRF key) terbuang sepenuhnya oleh pemanggil. Akibatnya, setiap descriptor yang dipublikasikan ke HSDir berubah status dari sekadar dokumen publik menjadi **kebocoran kunci master secara langsung**: siapa pun yang mengunduh satu descriptor dapat memulihkan scalar kunci identitas, lalu memproduksi descriptor sah untuk periode waktu berikutnya atas nama alamat korban.

1 descriptor publik

cukup untuk memulihkan kunci master secara penuh

Temuan ini konsisten dengan pola insiden yang dilaporkan. Layanan lama dan besar seperti forum onion umumnya sudah memiliki alamat yang terbangun reputasi, sehingga operator cenderung memuat kunci Tor yang sudah ada ke dalam GoBalance alih-alih membuat layanan baru; justru jalur inilah yang rentan. Sebaliknya, layanan yang dibuat baru melalui generator konfigurasi GoBalance memakai format kunci PEM yang jalur penandatanganannya aman. Proof-of-Concept yang dijalankan secara lokal membuktikan bahwa kunci yang dipulihkan dari satu descriptor publik mampu menghasilkan tanda tangan yang identik byte-per-byte dengan tanda tangan kunci asli untuk periode waktu berikutnya, yang berarti pengambilalihan penuh alamat onion hanya membutuhkan koneksi biasa ke jaringan Tor.

Temuan	Lokasi Kode	Dampak	Severity
Nonce blinded signature konstan dan publik (kunci format Tor)	pkg/stem/util/ed25519.go; pkg/stem/descriptor/hidden_service.go	Pemulihan kunci identitas master dari descriptor publik; takeover domain onion	Kritis

Temuan	Lokasi Kode	Dampak	Severity
Sertifikat kunci Tor dimuat tanpa verifikasi chain dan kunci PRF dibuang	pkg/onionbalance/tored25519/tor_ed25519.go	Akar penyebab kegagalan kriptografi pada jalur Tor	Kritis (terkait)
Descriptor instance tidak diverifikasi tanda tangannya	pkg/onionbalance/instance/instance.go	Injeksi introduction point oleh HSDir jahat (termitigasi Tor)	Sedang
Sakelar random deterministik (needDeterministic) pada generator kunci	pkg/brand/brand.go	Jika diaktifkan, kunci master dapat diprediksi dari waktu pembuatan	Sedang (desain berisiko)
Branch patch1 tidak memperbaiki kerentanan utama	origin/patch1	Pengguna tetap rentan meski mengikuti branch perbaikan	Informasional

Tabel 1. Ringkasan temuan analisis keamanan GoBalance

2. Latar Belakang: OnionBalance dan GoBalance

Pada Tor onion service versi 3, alamat *.onion* 56 karakter bukanlah nama yang terdaftar di suatu otoritas, melainkan turunan langsung dari kunci publik ed25519 milik layanan (kunci identitas). Siapa pun yang memegang kunci privat identitas memegang kendali penuh atas alamat tersebut, karena hanya pemegangnya yang dapat membuat descriptor sah yang mengikat alamat itu ke sekumpulan introduction point. Dengan kata lain, kunci identitas adalah padanan dari kepemilikan domain, DNS, dan sertifikat sekaligus dalam satu berkas. Kehilangan atau kebocoran kunci ini berarti kehilangan domain, dan korban tidak memiliki mekanisme pengaduan apa pun untuk merebutnya kembali.

OnionBalance bekerja sebagai frontend: beberapa backend instance (masing-masing onion service biasa) mendaftarkan introduction point-nya, lalu frontend mempublikasikan satu descriptor gabungan yang ditandatangani oleh kunci master, sehingga satu alamat tunggal dilayani oleh banyak backend. Proses ini menggunakan skema key blinding resmi rend-spec-v3: untuk setiap periode waktu (yang ditentukan nilai SRV dari konsensus Tor, publik dan berganti setiap interval tertentu), kunci identitas menghasilkan blinded key dan blinded signature. Descriptor final berisi sertifikat yang mengikat descriptor signing key ke blinded key, lalu diunggah ke HSDir dan dapat diunduh siapa pun. Semua material tanda tangan buta itu, bersama pesan yang ditandatangani, tersedia untuk pub secara tak terbatas waktunya.

GoBalance mereplikasi logika OnionBalance versi Python (beserta pustaka stem) ke dalam Go, dengan daya tarik utama berupa keterbebasan dari dependensi Python dan kemudahan integrasi sebagai pustaka. Repositori ini pertama kali muncul pada Maret 2022 dengan commit terakhir pada Januari 2023, dan menyertakan jalur konfigurasi berbasis YAML serta alat pembangkit konfigurasi. Bagi operator layanan besar seperti forum onion, alat semacam ini menarik karena membagi beban trafik dan daya tahan tanpa harus mengubah alamat yang sudah dikenal pengguna. Justru kebutuhan mempertahankan alamat lama itulah yang mendorong operator memuat kunci Tor yang sudah ada, dan dengan demikian mengaktifkan jalur kode yang rentan.

3. Pemetaan Jalur Kunci Kriptografis

Jalur kunci dimulai dari berkas konfigurasi YAML yang menunjuk ke berkas kunci master. Fungsi *loadServiceKeys* pada `pkg/onionbalance/service/service.go` mencoba membaca kunci tersebut sebagai PEM PKCS8 (format yang dihasilkan generator GoBalance/OnionBalance, memuat seed 32 byte); jika gagal, berkas dianggap sebagai kunci biner Tor dan dilewatkan ke *tored25519.LoadTorKeyFromDisk*, yang membuang header 32 byte dan mengambil 64 byte sisanya: 32 byte secret scalar (a) dan 32 byte PRF key (h). Format inilah yang dipakai Tor untuk menyimpan kunci privat hidden service di direktori data, dan inilah format yang dimiliki hampir semua layanan onion yang sudah berjalan lama sebelum mengadopsi GoBalance.

Komponen	Berkas	Peran
Muat kunci master	<code>pkg/onionbalance/service/service.go</code> (<i>loadServiceKeys</i>)	Memilih format PEM (seed) atau Tor (a h); menandai <i>isPrivKeyInTorFormat</i>
Pembungkus kunci	<code>pkg/gobpk/gobpk.go</code>	Wrapper <i>ed25519</i> ; <i>Seed()</i> mengembalikan 32 byte pertama untuk kedua format
Dispatcher signing	<code>pkg/stem/descriptor/hidden_service.go</code> (<i>blindedSign</i>)	Memilih <i>BlindedSign</i> atau <i>BlindedSignWithTorKey</i> berdasarkan format kunci
Matematika blinding	<code>pkg/stem/util/ed25519.go</code>	<i>BlindedSign</i> , <i>BlindedSignWithTorKey</i> , <i>blindedSignP2</i> , <i>BlindedPubkey</i>
Sertifikat descriptor	<code>pkg/stem/descriptor/certificate.go</code>	Membangun dan menandatangani <i>Ed25519CertificateV1</i> (cert tipe 8)
Publikasi	<code>pkg/onionbalance/descriptor/descriptor.go</code> ; <code>service.go</code>	Membangun <i>OBDescriptor</i> , menghitung revision counter, unggah via <i>HSPOST</i>

Tabel 2. Pemetaan komponen penting pada jalur kunci dan penandatanganan

Pada saat publikasi descriptor, fungsi *getSigningCert* membangun sertifikat tipe 8 yang mengikat descriptor signing key (dibuat acak per descriptor) ke blinded key, lalu meminta *blindedSign* menandatangani dengan kunci identitas master. Dispatcher pada `hidden_service.go` memilih fungsi signing berdasarkan format kunci: jalur PEM memakai *BlindedSign*, sedangkan jalur Tor memakai *BlindedSignWithTorKey*. Kedua jalur bertemu di *blindedSignP2* yang melakukan blinding scalar lalu menandatangani pesan ala Ed25519. Hasil tanda tangan (R, S) tertanam di dalam sertifikat descriptor yang kemudian diunggah ke *HSDir*, dapat diunduh oleh siapa pun, dan tetap tersedia selama descriptor hidup.

4. Kerentanan Kritis: Nonce Deterministik Konstan

Kerentanan berakar pada ketidakcocokan kontrak antara pemanggil dan fungsi signing. Komentar kode menyatakan *BlindedSignWithTorKey* menerima kunci yang sudah diperluas (extended key) 64 byte berformat Tor, dan fungsinya memang dirancang hanya melewati tahap ekspansi kunci. Namun dispatcher *blindedSign* mengirim *identityKey.Seed()*, yang untuk *gobpk.PrivateKey* berformat Tor hanya mengembalikan 32 byte pertama, yaitu secret scalar *a*. Separuh kunci yang seharusnya menjadi PRF key (*h*, penentu nonce) terbuang sejak titik ini. Di dalam *blindedSignP2*, potongan *esk[32:]* menjadi irisan kosong berukuran nol byte, sehingga input hash untuk derivasi *kPrime* berubah menjadi konstanta literal string yang sama untuk seluruh pengguna GoBalance berformat Tor di seluruh dunia.

4.1 Akar masalah pada kode

Tiga potongan kode berikut membentuk rantai kegagalan. Pertama, dispatcher memanggil fungsi signing dengan *identityKey.Seed()*, yang untuk kunci berformat Tor hanya 32 byte pertama (secret scalar *a*). Kedua, *BlindedSignWithTorKey* memanggil *Seed()* sekali lagi atas input 32 byte itu, sehingga *esk* yang diteruskan tetap 32 byte. Ketiga, *blindedSignP2* mengambil *esk[32:]* sebagai PRF key, yang kini berukuran nol byte.

```
func blindedSign(msg []byte, identityKey gobpk.PrivateKey,
    blindedKey, blindingNonce []byte) []byte {
    signFn := utils.Ternary(identityKey.IsPrivKeyInTorFormat(),
        util.BlindedSignWithTorKey, util.BlindedSign)
    return signFn(msg, identityKey.Seed(), blindedKey, blindingNonce)
}
```

```
// BlindedSignWithTorKey expects an EXTENDED (tor format) key:
// a||h, 64 bytes. It skips only the "expand the key" step.
func BlindedSignWithTorKey(msg []byte, identityKey ed25519.PrivateKey,
    blindedKey, blindingNonce []byte) []byte {
    esk := identityKey.Seed() // BUG: 64-byte key - Seed() = 32 bytes!
    return blindedSignP2(esk, msg, blindedKey, blindingNonce)
}
```

```
func blindedSignP2(esk, msg, blindedKey, blindingNonce []byte) []byte {
    // ... mult dihitung dari blindingNonce (publik, dari SRV) ...
    s := decodeInt(esk[:32])
    sPrime := biMod(biMul(s, mult), l)
    k := esk[32:] // <-- KOSONG (0 byte) pada jalur Tor
    tmp := sha512.Sum512([]byte("Derive temporary signing key hash input"
        + string(k))) // <-- KONSTAN untuk semua pengguna
    kPrime := tmp[:32]
    blindedEsk := append(encodeInt(sPrime), kPrime...)
    // ... r := hint(kPrime || msg); R = rB;
    // S = (r + H(R||PK||msg) * s') mod l ...
}
```

4.2 Aljabar pemulihan kunci

Dalam Ed25519, keamanan tanda tangan bertumpu pada ketakterdugaan nonce $r = H(\text{prefix} \parallel M)$; prefix adalah 32 byte rahasia milik kunci. Jika r diketahui, persamaan verifikasi $S = (r + H(R \parallel PK \parallel M) * s) \bmod L$ dapat dibalik langsung untuk memperoleh scalar s , karena semua komponen lain publik. Di jalur Tor GoBalance, prefix (kPrime) adalah konstanta publik yang di-hardcode implisit di kode, sehingga r dapat dihitung siapa pun dari pesan sertifikat yang ikut dipublikasikan. Scalar yang terbuka adalah scalar buta $s' = s * \text{mult} \bmod L$; namun multiplier blinding mult semata-mata diturunkan dari nonce blinding periode waktu yang berasal dari SRV konsensus Tor dan sepenuhnya publik. Dengan membalik mult , penyerang memperoleh $s = s' * \text{mult}^{-1} \bmod L$, yaitu scalar identitas master modulo orde kurva, yang secara kriptografis setara penuh dengan kunci master untuk semua operasi onion service.

$$\begin{aligned} r &= \text{SHA512}(\text{kPrime} \parallel M) \quad (\text{dapat dihitung siapa pun}) \\ s' &= (S - r) \cdot H(R \parallel PK \parallel M)^{-1} \bmod L \quad (\text{pemulihan scalar buta}) \\ s &= s' \cdot \text{mult}^{-1} \bmod L \quad (\text{unblinding; mult publik dari SRV}) \end{aligned}$$

Data	Sumber	Status
Bytes sertifikat (M) dan tanda tangan (R, S)	Descriptor di HSDir	Publik
Blinded key (PK) pada ekstensi HAS_SIGNING_KEY	Sertifikat descriptor	Publik
$\text{kPrime} = \text{SHA512}(\text{"Derive temporary signing key hash input"})$	Konstanta kode gobalance	Publik (konstanta)
Nonce $r = \text{SHA512}(\text{kPrime} \parallel M)$	Dihitung penyerang	Dapat dihitung publik
mult (multiplier blinding periode)	SRV konsensus Tor	Publik
Scalar identitas master (a)	Hasil pemulihan	RAHASIA - bocor
PRF key (h) asli	Dibuang oleh kode	Tidak diperlukan penyerang

Tabel 3. Pemisahan data publik dan data rahasia dari sudut pandang penyerang

4.3 Karakteristik serangan

Penting dicatat bahwa tanda tangan yang dihasilkan jalur cacat ini tetap **valid** menurut verifikasi Tor: R dan S memenuhi persamaan verifikasi, sehingga tidak ada alarm, penolakan descriptor, maupun anomali yang terlihat oleh klien maupun operator. Kerentanan berjenis ini (nonce deterministik yang dapat diprediksi) secara historis identik dengan keluarga kegagalan RNG pada ECDSA, namun di sini lebih buruk karena nonce bukan sekadar berulang secara kebetulan, melainkan bisa dihitung langsung oleh penyerang dari konstanta kode sumber. Satu tanda tangan cukup; tidak ada batas bawah jumlah sampel yang diperlukan.

5. Validasi Proof-of-Concept

Untuk memvalidasi temuan, dibangun proof-of-concept yang dijalankan sepenuhnya secara lokal terhadap kode repositori asli tanpa modifikasi, menggunakan kunci ed25519 yang dibangkitkan acak khusus untuk pengujian dan tidak menargetkan layanan nyata mana pun. Skenario pengujian mensimulasikan korban dengan kunci berformat Tor ($a||h$) yang menandatangani sertifikat descriptor melalui jalur kode persis seperti dipakai saat produksi, lalu menyimulasikan penyerang yang hanya memegang data publik: bytes sertifikat, (R , S), blinded key, dan nonce blinding publik. Sebagai kontrol, pengujian yang sama dijalankan pada jalur kunci PEM (seed) untuk memastikan kegagalan pemulihan di jalur yang seharusnya aman.

Hasilnya sesuai prediksi analisis. Scalar buta s' yang dipulihkan penyerang identik dengan scalar buta yang dipakai korban (pemeriksaan internal menunjukkan kesamaan penuh), dan setelah unblinding diperoleh scalar master modulo L . Langkah verifikasi terakhir adalah yang paling menentukan: penyerang menggunakan scalar hasil pemulihan untuk membangun tanda tangan buta bagi periode waktu baru (nonce blinding berbeda), dan hasilnya **identik byte-per-byte** dengan tanda tangan yang dihasilkan kunci asli korban pada periode yang sama. Artinya, kepemilikan scalar modulo L adalah kepemilikan efektif atas identitas layanan: penyerang dapat menurunkan blinded key untuk semua periode mendatang dan menandatangani descriptor yang sah di mata jaringan Tor.

Pengujian kontrol pada jalur PEM gagal memulihkan scalar, sebagaimana mestinya, karena pada jalur tersebut PRF key diturunkan dari ekspansi SHA-512 atas seed yang tetap rahasia, sehingga nonce tidak dapat dihitung pihak luar. Kontras kedua jalur ini mengisolasi akar masalah secara presisi: kerentanan bukan terletak pada matematika blinding (yang mengikuti spesifikasi `rend-spec-v3` dan implementasi stem), melainkan pada penanganan format kunci Tor yang membuang komponen nonce. Skrip pengujian tersimpan sebagai artefak lokal bersama salinan repositori analisis (`pkg/stem/util/attack_poc_test.go`) dan sengaja tidak dipaketkan sebagai alat eksploitasi siap pakai.

6. Temuan Sekunder

Selain kerentanan utama, analisis statis menemukan beberapa kelemahan pendukung yang memperbesar permukaan risiko operasional. Pertama, `pkg/brand/brand.go` menyediakan hack random deterministik melalui konstanta `needDeterministic`; saat ini bernilai `false` sehingga kunci dibangkitkan dari `crypto/rand`, namun cukup satu perubahan baris untuk membuat generator kunci master memakai `math/rand` berseed waktu sistem, yang ruang pencariannya bisa dippersempit ke menit pembuatan. Kedua, `RegisterDescriptor` pada `instance.go` menerima descriptor instance tanpa verifikasi tanda tangan maupun validitas rantai sertifikat; `HSDir` jahat yang menyajikan descriptor palsu bisa menyuntik introduction point pilihannya ke descriptor frontend korban. Tor sendiri memverifikasi descriptor saat mengunduh sehingga vektor ini termitigasi sebagian, tetapi kepercayaan buta semacam ini tetap menjadi cacat desain pertahanan berlapis.

Temuan	Lokasi	Risiko	Berat
Sakelar random deterministik pada generator kunci	pkg/brand/brand.go	Kunci master dapat diprediksi bila flag diaktifkan	Sedang
Tidak ada verifikasi tanda tangan descriptor instance	pkg/onionbalance/instance/instance.go	Injeksi introduction point oleh HSDir jahat (termitigasi Tor)	Sedang
math/rand berseed waktu untuk pengacakan urutan intro point	onionbalance.go; descriptor.go	Pengacakan dapat diprediksi; bukan kerentanan langsung	Rendah
Kunci master disimpan sebagai berkas polos tanpa enkripsi	config; service.go	Kompromi host frontend = kompromi seluruh identitas	Sedang
Branch patch1 hanya refaktor, tidak menyentuh cacat nonce	origin/patch1	Pengguna yang berpindah branch merasa aman padahal tidak	Tinggi (persepsi)

Tabel 4. Temuan sekunder beserta penilaian risikonya

Ketiga, seluruh materi kunci privat disimpan dalam berkas polos tanpa perlindungan tambahan, dan pemisahan antara frontend (yang memegang kunci master) dan backend tidak diikuti pengerasan apa pun pada host frontend: kontrol port Tor diakses dengan kata sandi polos melalui jaringan lokal. Keempat, branch patch1 yang berisi perbaikan race dan refaktor besar ternyata tetap memanggil *BlindedSignWithTorKey* dengan cara yang sama persis, sehingga pengguna yang berpindah ke branch perbaikan justru mendapat rasa aman palsu. Secara keseluruhan, pola-pola ini menunjukkan kode ditulis untuk kebenaran fungsional tanpa review kriptografi yang menyeluruh, sesuatu yang wajib dianggap standar minimum untuk perangkat yang mengelola identitas digital bernilai seperti kunci master onion service.

7. Rekonstruksi Skenario Pengambilalihan Domain

Menggabungkan temuan di atas dengan konteks insiden, rekonstruksi berikut menjelaskan mengapa dua forum besar menjadi korban dan mengapa pengambilalihan dapat berlangsung mulus. Profil korban sesuai dengan pengguna jalur rentan: forum besar hampir selalu merupakan layanan lama dengan alamat yang sudah melekat pada komunitasnya. Ketika mengadopsi GoBalance untuk load balancing, operator memuat kunci `hs_ed25519_secret_key` milik alamat lama tersebut, dan sejak detik itu setiap descriptor yang dipublikasikan adalah kebocoran kunci yang menunggu dibaca. Tidak perlu ada kesalahan opsec, phishing, atau malware pada operator; memakai alat seadanya sudah cukup.

7.1 Langkah serangan end-to-end

1. Penyerang mengunduh descriptor frontend korban dari HSDir mana pun (operasi baca publik).
2. Sertifikat descriptor-signing-key-cert diurai: diperoleh bytes sertifikat M, tanda tangan (R, S), dan blinded key PK.
3. Dengan kPrime konstan dari kode sumber, penyerang menghitung $r = \text{SHA512}(k\text{Prime} \parallel M)$.

4. Scalar buta dipulihkan: $s' = (S - r) \cdot H(R||PK||M)^{-1} \bmod L$, lalu di-unblind dengan mult dari SRV periode terkait.
5. Untuk periode waktu berikutnya, penyerang menurunkan blinded key baru dan membangun descriptor dengan introduction point miliknya.
6. Descriptor palsu diunggah via HSPOST dari koneksi Tor biasa; jaringan menerimanya karena tanda tangan sah.
7. Klien yang mencari alamat korban menerima descriptor penyerang dan terhubung ke server pengganti: alamat dialihkan.

Rangkaian langkah itu tidak pernah menyentuh server backend korban, tidak memerlukan exploit memori, dan tidak menyisakan jejak log pada infrastruktur korban; satu-satunya prasyarat adalah ketertarikan pada alamat target dan koneksi Tor. Kronologi yang Anda sampaikan, kedua forum dialihkan ke domain onion lain, sangat cocok dengan mekanisme ini: alamat lama tetap hidup, tetapi descriptor-nya kini menunjuk ke introduction point milik pihak pengambil alih, sehingga pengunjung tetap bisa masuk namun tiba di tiruan atau pengganti milik penyerang. Alternatifnya, operator yang menyadari kebocoran akan bergegas membuka alamat baru dan mengumumkan migrasi, yang dari luar juga tampak sebagai pengalihan domain.

Perlu ditegaskan bahwa analisis ini tidak melakukan atribusi. Kemungkinan penyebab lain seperti kompromi host frontend, pencurian berkas kunci, atau rekayasa sosial tetap mungkin secara umum; namun kerentanan yang dibahas di sini penting karena meniadakan prasyarat-prasyarat tersebut. Sejak repositori dipublikasikan (2022) hingga kini, flaw ini bisa ditemukan siapa pun yang membaca kode dengan cermat, atau bahkan ditemukan secara empiris tanpa membaca kode sama sekali. Dua forum besara yang terdampak bersamaan juga konsisten dengan penyerang tunggal yang memindai descriptor onionbalance berformat Tor secara massal dan mengotomatiskan pemulihan kunci untuk banyak target sekaligus.

8. Rekomendasi Remediasi

Rekomendasi dibagi untuk tiga audiens: operator yang pernah atau sedang memakai GoBalance, pengembang perangkat lunak onionbalance turunan, dan komunitas keamanan Tor secara umum. Untuk operator, asumsi yang benar adalah kunci master yang pernah dipakai dengan GoBalance berformat Tor harus dianggap telah bocor sejak descriptor pertamanya dipublikasikan; rotasi identitas ke alamat baru dibangun dari kunci segar adalah satu-satunya remediasi yang benar-benar menutup celah, karena mengganti server atau kata sandi tidak mengembalikan kendali atas descriptor. Selama masa migrasi, pengumuman alamat baru harus disebarkan melalui kanal yang tidak bergantung pada alamat lama yang sudah dapat dibajak kapan pun.

Prioritas	Aksi	Penjelasan
P0 - Operator	Rotasi identitas: alamat onion baru dari kunci segar; migrasi terkendali	Kunci lama dianggap bocor permanen; perbaikan di sisi alat tidak mengembalikan kendali

Prioritas	Aksi	Penjelasan
P0 - Pengembang	Kirim kunci 64 byte utuh ($a h$) ke <code>BlindedSignWithTorKey</code> agar <code>esk[32:] = h</code>	Perbaikan satu baris pada pemanggil; wajib diuji ulang dengan test vector stem/tor
P1 - Pengembang	Tambahkan verifikasi tanda tangan descriptor instance sebelum diterima	Pertahanan berlapis terhadap HSDir jahat; jangan bergantung pada verifikasi Tor
P1 - Pengembang	Hapus jalur random deterministik (pkg/brand) dari kode produksi	Generator kunci tidak boleh memiliki sakelar ke <code>math/rand</code> sekecil apa pun
P2 - Komunitas	Audit silang alat turunan onionbalance lain; umumkan advisory	Pola cacat sering tersalin antar-reimplementasi; pengguna perlu tahu statusnya
P2 - Operator	Pantau anomali descriptor (rev counter melonjak, intro point asing)	Deteksi dini pembajakan descriptor sebelum dampak menyebar ke pengguna

Tabel 5. Rekomendasi remediasi berdasarkan prioritas

Untuk pengembang, perbaikan minimal adalah mengubah pemanggil pada dispatcher agar meneruskan kunci Tor utuh 64 byte sehingga `esk[32:]` berisi PRF key yang sebenarnya, mengembalikan nonce ke posisi yang tak terduga; perubahan ini wajib disertai vektor uji yang membandingkan keluaran dengan implementasi referensi stem agar kompatibilitas tanda tangan terjaga. Di atas perbaikan titik, disarankan mengganti seluruh implementasi `ed25519` buatan sendiri dengan pustaka standar yang teruji, karena berkas-berkas di `pkg/stem/util` dan `pkg/onionbalance/hs_v3/ext` adalah salinan manual implementasi referensi yang rapuh terhadap kesalahan transkripsi. Pelajaran terbesar untuk ekosistem adalah proses: perangkat yang mengelola kunci identitas harus melewati review kriptografi independen sebelum dipromosikan ke pengguna produksi.

9. Lingkup, Etika, dan Batasan

Analisis ini dilakukan sebagai riset keamanan defensif dan forensik. Seluruh pengujian proof-of-concept dijalankan lokal terhadap kunci yang dibangkitkan sendiri untuk keperluan pengujian; tidak ada pengujian, pengambilan descriptor, maupun interaksi terhadap layanan onion nyata mana pun selama pengerjaan. Laporan ini sengaja menjelaskan mekanisme kerentanan pada tingkat yang cukup untuk verifikasi teknis dan perbaikan kode, tanpa menyertakan alat eksploitasi siap pakai. Bila repositori tetap aktif, disarankan melakukan responsible disclosure kepada pemilik repositori dan, karena dampaknya luas, kepada kanal keamanan komunitas Tor agar pengguna yang rentan dapat diberi tahu.

Terdapat beberapa batasan yang perlu dipahami pembaca. Pertama, analisis dilakukan terhadap kode sumber cabang master dan `patch1` per commit terakhir yang tersedia publik (Januari 2023); biner rilis pihak ketiga dapat berbeda. Kedua, korelasi antara kerentanan ini dan insiden pengambilalihan dua forum bersifat indikatif berdasarkan kesesuaian mekanisme dan profil korban, bukan hasil investigasi forensik atas artefak insiden yang sesungguhnya, yang berada di luar akses kami. Ketiga, estimasi dampak mengasumsikan implementasi mengikuti kode yang dianalisis tanpa modifikasi lokal oleh operator. Dengan batasan tersebut, temuan teknis inti telah diverifikasi secara deterministik melalui proof-of-concept dan dapat direproduksi siapa pun dari

kode sumber publik.